DHBW Heilbronn

Software Engineering II.1 – Block 5

Testen

Risiken systematisch in aussagekräftige Tests übersetzen

Software Engineering II.1 | WS 2026/27 | Dr. rer. nat. Ana Nicolaescu

Wo stehen wir im Kurs?

Block 1	Block 2	Block 3	Block 4	Block 5	Block 6	Block 7
4 UE	4 UE	4 UE		8 UE		4 UE
Einführung & Vorgehensmodelle	Requirements Engineering	Entwurf & Entwurfsmuster	Implementierung & Versionsmanagement	Testen (Blackbox)	Whitebox, Wartung & Change Request	Management & Abschlusspräsentation

Durchgängiges Gruppenprojekt „Photo Album“: jeder Block liefert ein prüfbares Artefakt ins Repository

Teams, Repo, Vorgehen	REQUIREMENTS.md Use Cases · AK · Prototyp	ARCHITECTURE.md Klassen- · Sequenzdiagramm	Build · Git · PR Review · CI grün	TESTING.md Unit/Integr. · Blackbox	Coverage · Refactoring Change Request	RISK_ANALYSIS.md Präsentation

Heute werden Anforderungen zu Testfällen

- Verifikation vs. Validierung; statisch vs. dynamisch; Teststufen und Testpyramide
- Was einen guten Testfall ausmacht: Absicht, AAA, sprechender Name
- Blackbox-Verfahren: Äquivalenzklassen, Grenzwertanalyse, Entscheidungstabellen, Zustandsmodelle – jedes am Photo Album
- Integrationstests, die etwas Sinnvolles prüfen; Usability-, Security- und Performance-Tests
- Testen von LLMs und Agenten: Evaluation Sets, Nichtdeterminismus, Halluzinationen

Projektergebnis dieses Blocks
TESTING.md: Teststrategie + dokumentiertes Blackbox-Verfahren
≥ 10 Unit-Tests mit Normal-, Fehler-, Grenzfällen
≥ 2 Integrationstests (Use Case + Repository)
Coverage-Report erzeugt (Interpretation: Block 6)
Mini-Evaluation-Set für eine KI-Funktion

Einstieg: Würden Sie releasen?

Übung

Statusbericht eines Teams: „Alle 37 Tests sind grün. Coverage: 92 %. Wir sind fertig.“

5 min

Team

AUFTRAG

1. Abstimmung im Team: Release ja/nein? (1 min)
2. Notieren Sie drei Informationen, die Ihnen für die Entscheidung fehlen (3 min).
3. Formulieren Sie die eine Frage, die Sie dem Team stellen würden (1 min).

ERGEBNIS Drei fehlende Informationen + eine Frage pro Team.

DANACH IM PLENUM Handzeichen ja/nein; drei Fragen sammeln; ich verrate, was die 37 Tests wirklich prüfen.

1

Grundbegriffe und Teststufen

Was prüfen wir wogegen – und auf welcher Ebene?

Verifikation, Validierung, statisch, dynamisch

Verifikation – bauen wir es richtig?

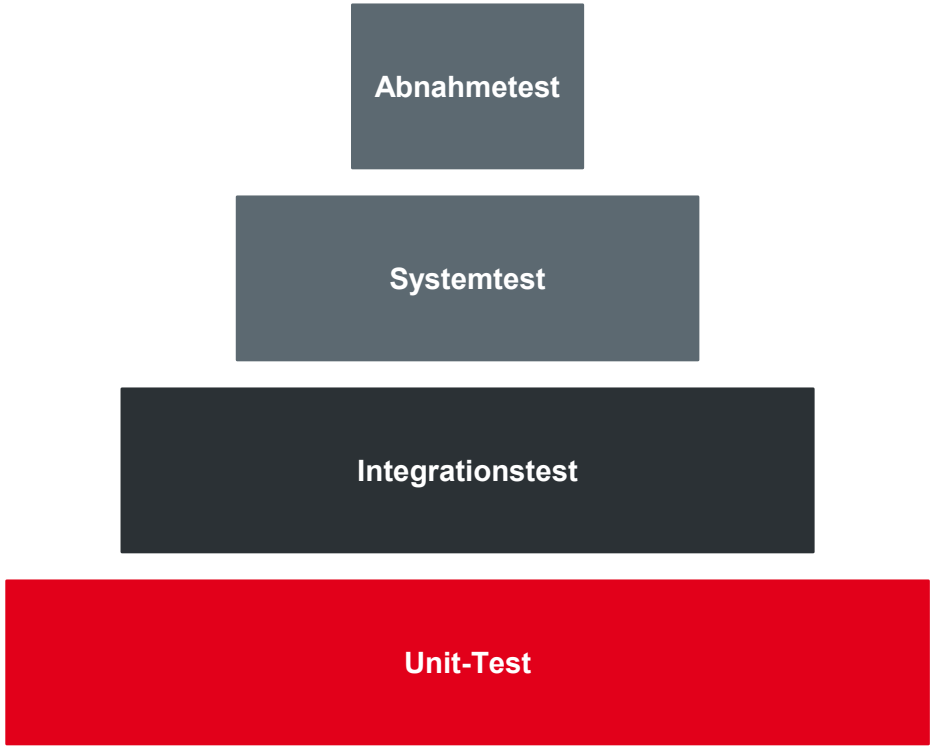
- Prüfung gegen die Spezifikation: erfüllt der Code FA-05 und AK-05.1?
- Mittel: Reviews (statisch), Unit-/Integrationstests (dynamisch), statische Analyse
- Statisch = ohne Ausführung: Code Review (Block 4), Checkstyle/SpotBugs, Anforderungs-Review (Block 2)
- Dynamisch = mit Ausführung: Testdaten + Orakel (erwartetes Ergebnis) + beobachtetes Verhalten

Validierung – bauen wir das Richtige?

- Prüfung gegen den Bedarf: Löst das Photo Album das Problem der Nutzerin?
- Mittel: Abnahme durch den Auftraggeber, Usability-Test, Prototyp-Feedback (Block 1/2)
- Ein System kann verifiziert und trotzdem falsch sein: alle Tests grün, Nutzerin findet den Löschen-Knopf nicht
- Ihr Peer Review in Block 2 war Validierung; Ihre Unit-Tests heute sind Verifikation

Dijkstra: „Testen kann die Anwesenheit von Fehlern zeigen, nie ihre Abwesenheit.“ Deshalb: Risiken priorisieren, nicht Vollständigkeit vortäuschen.

Teststufen und Testpyramide: ein Kostenmodell



← teuer, langsam, realistisch

schnell, isoliert, viele →

Auftraggeber prüft gegen Akzeptanzkriterien – manuell, Präsentation

gesamte Anwendung über UI: Album anlegen → Foto hinzufügen → Neustart → noch da?

Use Case + echtes FileAlbumRepository + temporäres Verzeichnis; Duplikat nach Neustart erkannt?

Album.addPhoto: 99/100/101 Fotos, Duplikat, leerer Name – ohne Datei, ohne UI, in Millisekunden

Projekt: ≥ 10 Unit (Domäne + Use Cases mit Fakes) · ≥ 2 Integration (Use Case + echte Infrastruktur) · 1–2 manuelle Systemtests dokumentiert · Abnahme = Präsentation

Testarten folgen dem Risiko

Testart	Frage	Photo Album
Funktional		
Regression		
Sicherheit		
Performance		
Usability		
Kompatibilität		

Testarten folgen dem Risiko

Testart	Frage	Photo Album
Funktional	Tut es, was die Anforderung sagt?	alle FA/AK – heute
Regression	Funktioniert Altes nach Änderung noch?	gesamte Suite in CI bei jedem PR; Change Request (Block 6)
Sicherheit	Was passiert bei bösartiger Eingabe?	Datei mit .jpg-Endung, aber PNG-Inhalt; Pfad ../../etc; 200-MB-Datei
Performance	Wie schnell unter welcher Last?	NFA: Albumansicht mit 100 Fotos ≤ 2 s – messen, nicht schätzen
Usability	Kann eine echte Person die Aufgabe lösen?	Kreuztest heute: Album anlegen ohne Anleitung
Kompatibilität	Läuft es dort, wo es laufen soll?	CI auf Linux, Entwickler auf Windows/macOS – Pfadtrennzeichen!

Ein guter Testfall erklärt seine Absicht: Arrange – Act – Assert

```
class AlbumTest {

    @Test
    void rejectsDuplicatePhoto_andLeavesAlbumUnchanged() {   // AK-05.1
        // Arrange
        Album album = new Album("Urlaub 2026");
        Photo photo = new Photo(PhotoId.next(), "abc123hash");
        album.addPhoto(photo);

        // Act + Assert (Fehlerfall)
        assertThrows(DuplicatePhotoException.class,
                     () -> album.addPhoto(photo));

        // Assert (Zustand unverändert!)
        assertEquals(1, album.photos().size());
    }

    @ParameterizedTest
    @ValueSource(strings = {"", "   ", "\t"})
    void rejectsBlankName(String name) {                        // AK-01.1
        assertThrows(InvalidAlbumNameException.class, () -> new Album(name));
    }
}
```

- Name = Verhalten + Bedingung, nicht test1(); der AK-Bezug steht im Kommentar oder @DisplayName
- Arrange: nur, was der Test braucht; Act: genau eine Aktion; **Assert: Ergebnis UND Zustand**
- Fehlerfall ohne Zustandsprüfung ist ein halber Test (Einstiegsbeispiel!)
- Ein Test, eine Absicht – bricht er, weiß man sofort, welche Regel verletzt ist

2

Blackbox-Verfahren I

Äquivalenzklassen und Grenzwertanalyse: weniger Testdaten, mehr Fehler.

Äquivalenzklassen reduzieren Testdaten auf Repräsentanten

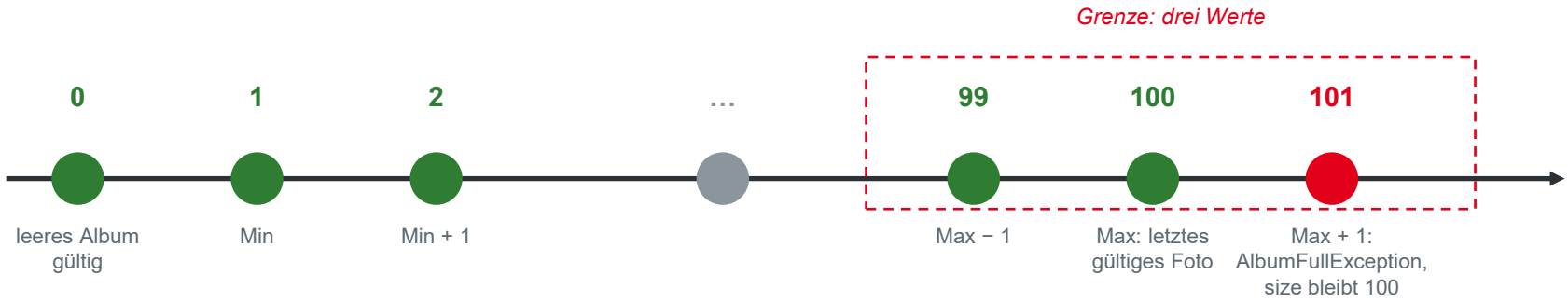
Regel FA-01: Der Albumname muss 1 bis 50 Zeichen lang sein und darf nicht nur aus Leerzeichen bestehen.

Klasse	Typ	Beschreibung	Repräsentant	Erwartetes Verhalten
ÄK1	**gültig**	1–50 Zeichen, mindestens ein Nicht-Leerzeichen	„Urlaub 2026“	Album angelegt
ÄK2	**ungültig**	leer (0 Zeichen)	„“	InvalidAlbumNameException, kein Album
ÄK3	**ungültig**	nur Leerzeichen/Whitespace	„ “	InvalidAlbumNameException
ÄK4	**ungültig**	länger als 50 Zeichen	51 × „a“	InvalidAlbumNameException
ÄK5	**ungültig**	null	null	InvalidAlbumNameException (nicht NPE!)
ÄK6	**?**	Sonderzeichen, Emoji, führende Leerzeichen	„ 🎉 Party “	Auftraggeber fragen → dokumentieren

Ergebnis: 6 Tests statt unendlich vieler – und eine offene Fachfrage, die vorher eventuell niemand gesehen hat.
Solche Tabellen in TESTING.md dokumentieren!

Grenzwertanalyse prüft die Übergänge

Regel FA-06: Ein Album darf höchstens 100 Fotos enthalten.



- Gültiger Bereich [1, 100] → Grenzen bei 1 und 100 → Werte 0, 1, 2 und 99, 100, 101
- Technische Grenzen (Integer.MAX_VALUE, 20 MB in Bytes) und fachliche Grenzen (100 Fotos) unterscheiden
- Jeder Grenzwert wird ein Test; der Test für 101 prüft Exception UND unveränderten Zustand
- Kombiniert mit Äquivalenzklassen: Repräsentant aus der Mitte + Grenzwerte an den Rändern

Faustregel: Wo eine Zahl in der Anforderung steht, gibt es drei Tests.

Übung: Äquivalenzklassen und Grenzwerte für den Fototitel

Übung

Regel FA-08 (Auftraggeber, Block 2): Ein Fototitel ist optional und darf höchstens 100 Zeichen lang sein. Die Beschreibung ist optional und darf höchstens 500 Zeichen lang sein.

15 min

Paare

AUFTRAG

1. Bilden Sie die Äquivalenzklassen für den Titel (gültig/ungültig/unklar) mit je einem Repräsentanten.
2. Bestimmen Sie die Grenzwerte für Titel und Beschreibung.
3. Notieren Sie die Fachfrage, die dabei entsteht (Tipp: „optional“ und „leer“).

ERGEBNIS Tabelle wie Folie 12 + Grenzwertliste; mindestens eine offene Fachfrage.

DANACH IM PLENUM Ein Paar nennt seine Fachfrage; ich antworte als Auftraggeber.

3

Blackbox-Verfahren II

Entscheidungstabellen und Zustandsmodelle: Kombinationen und Reihenfolgen.

Entscheidungstabellen prüfen Kombinationen von Bedingungen

Use Case „Foto hinzufügen“ – Bedingungen aus FA-04 (Dateityp), FA-06 (max. 100), FA-05 (Duplikat)

	R1	R2	R3	R4
B1 Dateityp gültig (JPEG/PNG, ≤ 20 MB)	N	J	J	J
B2 Album hat < 100 Fotos	–	N	J	J
B3 Foto bereits im Album (Hash)	–	–	J	N
A1 Foto speichern und anzeigen				X
A2 Fehler „ungültige Datei“	X			
A3 Fehler „Album voll (100/100)“		X		
A4 Fehler „bereits enthalten“			X	
A5 Album unverändert	X	X	X	

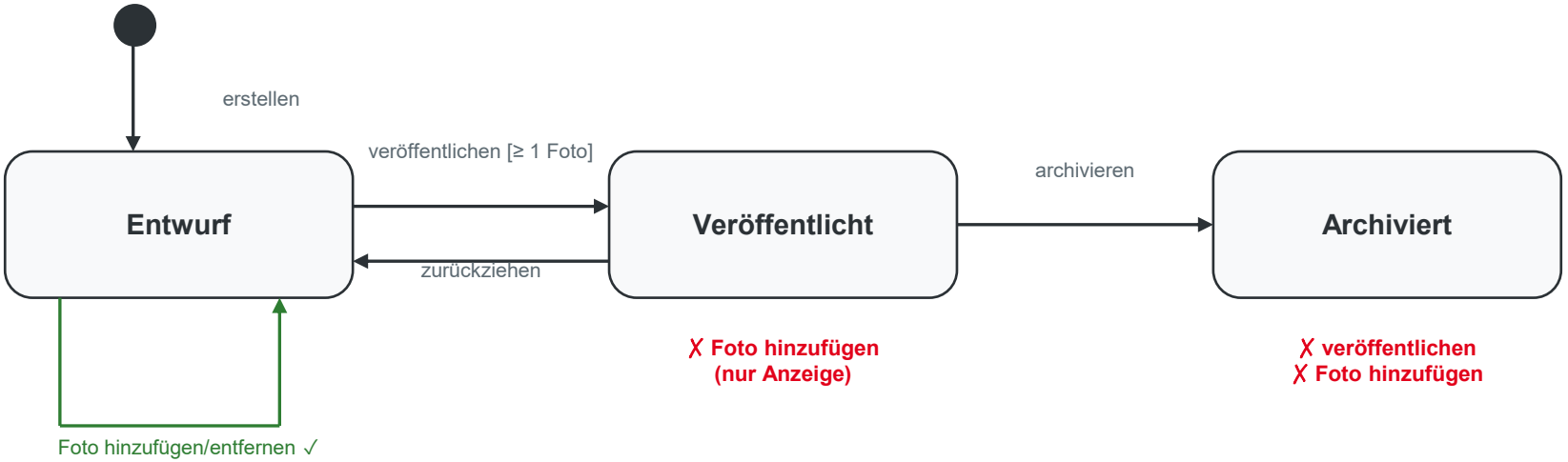
- Bx: Bedingung
- Ay: Aktion
- Rz: Regel

- „–“ = irrelevant: Prüfung findet nicht statt → Regeln kollabieren von 8 auf 4
- **Jede Spalte = ein Testfall** mit Namen: R2 → rejectsPhotoWhenAlbumFull()
- **Reihenfolge der Bedingungen ist eine Fachentscheidung** (welche Fehlermeldung bei voll UND Duplikat?)
- Aktion A5 ist in jedem Fehlerfall zu prüfen – die häufigste Lücke
- Dokumentation in TESTING.md: Tabelle + Zuordnung Regel → Testmethode

Wann Entscheidungstabelle? Sobald zwei oder mehr Bedingungen das Ergebnis gemeinsam bestimmen.

Zustandsmodelle prüfen erlaubte Reihenfolgen

Hypothetische Erweiterung: Album-Lebenszyklus Entwurf → Veröffentlicht → Archiviert



- Tests für jeden erlaubten Übergang: publishedAlbum_canBeArchived()
- Tests für jeden verbotenen Übergang: publishedAlbum_rejectsAddPhoto() – meist vergessen!
- Guards testen: Entwurf ohne Fotos → veröffentlichen abgelehnt
- Ein Pfadtest durch alle Zustände (Erstellen → Foto → Veröffentlichen → Archivieren)

Wann Zustandsmodell? Sobald das Ergebnis einer Aktion davon abhängt, was vorher passiert ist.

Vier Verfahren, vier Fragen – welches wann?

Verfahren	Leitfrage	Typische Regel im Photo Album	Liefert
Äquivalenzklassen	Welche Eingaben verhalten sich gleich?	Albumname, Fototitel, Dateityp	Repräsentanten je Klasse, ungültige Klassen
Grenzwertanalyse	Wo steht eine Zahl?	max. 100 Fotos, 50 Zeichen, 20 MB	3 Werte je Grenze
Entscheidungstabelle	Wirken mehrere Bedingungen zusammen?	Foto hinzufügen: Typ × voll × Duplikat	eine Regel = ein Test, Reihenfolge
Zustandsmodell	Hängt das Ergebnis von der Vorgeschichte ab?	Album-Lebenszyklus, Bestätigungsdialog	Übergangs- und Verbotstests

Projektvorgabe: mindestens ein Verfahren dokumentiert. Empfehlung: zwei – Grenzwerte sind fast immer dabei, plus Entscheidungstabelle für „Foto hinzufügen“.

Test Design Lab

Teamarbeit

Wählen Sie eine Geschäftsregel und einen Use Case aus Ihrem REQUIREMENTS.md. Testentwurf + Implementierung in JUnit.

30 min

Team

AUFTRAG

1. Verfahren 1 (Äquivalenzklassen + Grenzwerte) auf die Regel anwenden: Tabelle in TESTING.md.
2. Verfahren 2 (Entscheidungstabelle oder Zustandsmodell) auf den Use Case anwenden: Tabelle/Diagramm in TESTING.md.
3. Jede Zeile/Regel/Übergang → ein JUnit-Test mit sprechendem Namen; Normal-, Fehler-, Grenzfall markieren (@Tag oder Kommentar).
4. Zuordnung dokumentieren: AK-ID → Verfahren → Testmethode (Traceability aus Block 2).
5. Branch, PR, CI grün. Review über Kreuz ist Hausaufgabe.

ERGEBNIS TESTING.md mit zwei dokumentierten Verfahren + ≥ 6 neue Tests im PR.

MATERIAL TESTING.md-Vorlage mit Tabellen (Moodle), JUnit-5-Spickzettel

Mutation ohne Werkzeug

Übung

Ändern Sie gedanklich (oder wirklich, auf einem Wegwerf-Branch) in Album.addPhoto eine Vergleichsoperation: `>=` wird zu `>`.

10 min

Team

AUFTRAG

1. Welcher Ihrer Tests schlägt jetzt fehl? Wenn keiner: Ihre Tests haben die Grenze nicht abgedeckt – ergänzen Sie den Test für 100/101.

2. Zweite Mutation: Löschen Sie die Duplikatprüfung. Welcher Test wird rot?

3. Dritte Mutation: Vertauschen Sie die Reihenfolge von Kapazitäts- und Duplikatprüfung. Merkt ein Test das? Sollte er?

Integrationstests, die etwas Sinnvolles prüfen

```
class AddPhotoIntegrationTest {

    @TempDir Path tempDir;                          // JUnit 5: frisches
Verzeichnis je Test

    @Test
    void duplicateIsDetectedAfterRestart() throws IOException {
        // Arrange: echte Infrastruktur, kein Fake
        var repo    = new FileAlbumRepository(tempDir);
        var hasher  = new Sha256ImageHasher();
        var addPhoto = new AddPhotoUseCase(repo, hasher);
        var albumId = new CreateAlbumUseCase(repo).execute("Urlaub");
        Path jpg = copyResource("sample.jpg", tempDir);
        addPhoto.execute(albumId, jpg);

        // Act: "Neustart" – neues Repository-Objekt liest von Platte
        var repoAfterRestart = new FileAlbumRepository(tempDir);
        var addPhotoAgain = new AddPhotoUseCase(repoAfterRestart, hasher);

        // Assert
        assertThrows(DuplicatePhotoException.class,
                     () -> addPhotoAgain.execute(albumId, jpg));
        assertEquals(1, repoAfterRestart.findById(albumId).photos().size());
    }
}
```

- Integrationstest = mehrere echte Komponenten zusammen; hier: Use Case + Datei-Repository + Hasher
- Prüft, was Unit-Tests nicht können: Persistenz, Serialisierung, Pfade
- @TempDir: isoliert, aufgeräumt, parallel ausführbar
- Zweiter sinnvoller Test: UI-Adapter → Use Case (ohne echtes Fenster, falls JavaFX: Controller-Logik)
- Kein sinnvoller Integrationstest: „Repository speichert und lädt“ ohne fachliche Frage

4

Usability, Security, Performance

Testarten, die Unit-Tests nicht abdecken – und heute trotzdem einmal durchgeführt werden.

Usability-Test beobachtet echte Nutzung – Security und Performance messen

Usability (heute im Kreuztest)

- Konkrete Aufgabe: „Legen Sie ein Album ‚Sommer' an und fügen Sie dieses Foto hinzu."
- Nicht erklären, nicht helfen – nur beobachten und notieren: Wo zögert die Person? Was klickt sie falsch?
- Beobachtungen statt Meinungen: „suchte 20 s den Hinzufügen-Knopf" statt „fand es unübersichtlich"
- Probleme nach Auswirkung priorisieren: Aufgabe nicht lösbar > verzögert > kosmetisch
- 5 Testpersonen finden ~80 % der Probleme (Nielsen)

Security und Performance (aus den NFA)

- Security: Datei mit falscher Endung, 0 Byte, 200 MB, Pfad mit ../, Dateiname mit Steuerzeichen – jeder Fall ein Test
- Bildmetadaten als Angriffsfläche (Block 2: Prompt Injection) – Adapter strippt sie? Test!
- Performance: NFA sagt „100 Fotos ≤ 2 s" – messen mit 100 echten Dateien, nicht schätzen; Ergebnis in TESTING.md
- Große Alben, viele Alben, parallele Aufrufe – was passiert bei 1000 Fotos? (Grenze außerhalb der Anforderung, aber Verhalten dokumentieren)

5

KI-Perspektive

Testen von LLMs und Agenten: Evaluation Sets, Nichtdeterminismus, Halluzinationen.

Deterministischer Test und LLM-Evaluation

Klassische Funktion (Album.addPhoto)

- Gleiche Eingabe → gleiche Ausgabe
- Ein erwartetes Ergebnis, eine harte Assertion: assertEquals
- Grün oder rot; ein Lauf genügt
- Fehler ist reproduzierbar

LLM-Funktion (Beschreibung vorschlagen)

- Gleiche Eingabe → variierende Ausgabe (Temperatur, Modellversion)
- Mehrere Qualitätskriterien statt eines Werts: Länge, Sprache, keine Personennamen, fachlich passend
- Schwellenwerte über viele Fälle: „≥ 85 % der Fälle bestehen alle Kriterien“
- Wiederholte Läufe; Fehler kategorisieren (Halluzination, Formatfehler, Verweigerung, Injection-Erfolg)

Konsequenz: Ein Evaluation Set ist die Test-Suite für KI-Komponenten – versioniert im Repository (Block 4), ausgeführt bei jeder Prompt-/Modelländerung (Regression).

Agententests prüfen auch Aktionen, nicht nur Antworten

- Tool-Auswahl: *Wählt der Agent das richtige Tool mit* den richtigen Argumenten? (movePhoto(id, album) statt deleteAlbum)
- Berechtigungsgrenzen: Versucht er ein Tool, das nicht in der Allow-List ist? Wird es *abgelehnt und protokolliert*?
- Abbruch und Human Approval: Bleibt er bei irreversiblen Aktionen stehen und fragt?
- Prompt Injection und Datenabfluss: Adversarial-Fälle mit Anweisungen in Daten – kein Tool-Aufruf darf daraus folgen
- Kosten und Terminierung: max. Tool-Aufrufe, Timeout, keine Endlosschleifen
- Umsetzung: Fake-Tools im Test (wie InMemoryRepository), Protokoll der Aufrufe wird gegen Erwartung geprüft

Mini-Evaluation-Set

KI-Perspektive

Angenommen, Ihr Photo Album schlägt per KI eine Beschreibung vor (auch wenn Sie es nicht bauen).

10 min

Team

AUFTRAG

1. Definieren Sie fünf Fälle – je einen: normal, mehrdeutig, sensibel, adversarial, fehlerhaft – mit konkreter Eingabe.
2. Legen Sie je Fall 2–3 prüfbare Kriterien fest (regelbasiert wo möglich).
3. Bestimmen Sie die Pass-Rate, ab der Sie das Feature freigeben würden – und begründen Sie die Zahl.

ERGEBNIS eval/describe-photo/cases.md im Repository (auch als Won't-Feature: zeigt Testkompetenz).

DANACH IM PLENUM Ein Team liest seinen adversarialen Fall vor.

Projektcheckpoint Block 5

Teamarbeit

Jedes Team bewertet sich selbst: grün = im Repository sichtbar · gelb = begonnen · rot = fehlt. Für jeden gelben/roten Punkt entsteht ein Issue mit Verantwortlichem.



TESTING.md: Teststrategie (Stufen, Arten) und zwei dokumentierte Blackbox-Verfahren mit Tabellen



≥ 10 Unit-Tests: Normal-, Fehler-, Grenzfälle markiert; jeder Fehlerfall prüft den Zustand



≥ 2 Integrationstests mit echter Infrastruktur (@TempDir) und fachlicher Frage



Traceability: AK-ID → Verfahren → Testmethode dokumentiert



Usability-Protokoll und 3 Issues; Security-Fälle (Dateien) als Tests geplant



(Optional) Mini-Evaluation-Set in eval/ vorhanden

Abgabe des Zwischenstands: Commit/Tag im Repository bis zum Vorabend des nächsten Blocks.

Exit Ticket und bis zum nächsten Block

Exit Ticket (3 Minuten, jede Person schriftlich)

- Welcher Ihrer Tests findet am wahrscheinlichsten einen echten Fehler – und warum?
- Welches Risiko Ihres Photo Albums ist noch ungetestet?
- Welche Zahl allein beweist keine Qualität – und was müsste dazukommen?

Bis zum nächsten Block (Teamarbeit)

- Tests vervollständigen: ≥ 10 Unit + ≥ 2 Integration, alle grün in CI; PR mit Review über Kreuz (Tag „block-5“)
- Security-Tests für Dateieingaben implementieren (falsche Endung, 0 Byte, zu groß)
- Performance-Messung für Ihre NFA durchführen und Ergebnis in TESTING.md notieren
- JaCoCo-Report ansehen: Notieren Sie drei Klassen mit niedrigster Branch Coverage
- Usability-Issues: mindestens eines beheben (PR)
- AI_USAGE.md Fall 4: KI Tests aus einem Akzeptanzkriterium generieren lassen; prüfen, ob die Assertions wirklich das Kriterium prüfen (oft nicht!)

Quellen und Materialbasis Block 5

- ISTQB Certified Tester Foundation Level Syllabus v4.0 – Teststufen, Testarten, Blackbox-Verfahren
- Spillner, A. & Linz, T.: Basiswissen Softwaretest – Äquivalenzklassen, Grenzwerte, Entscheidungstabellen, Zustandstests
- Khorikov, V.: Unit Testing Principles, Practices, and Patterns; JUnit 5 User Guide; pitest.org (Mutation Testing)
- Cohn, M.: Succeeding with Agile (Testpyramide); Nielsen Norman Group: Usability Testing 101
- OWASP File Upload Cheat Sheet; OWASP Top 10 for LLM Applications
- Anthropic: Building effective agents; Zheng et al. (2023): Judging LLM-as-a-Judge